Assignment 6: `unique`
Making and Using a
Binary-Tree Abstract Data Type

by Dr. Kerry Veenstra
CSE 13S, Fall 2025
Document version 1

Due Wednesday November 18th, 2026, at 11:30 pm

Contents

1 Introduction **2**
1.1 Get the Resources . . . 2
1.2 Run an Example . . . 2

2 About Binary Search Trees **3**
2.1 Searching a Binary Search Tree for a Key . . . 4
2.2 Adding a New Node to a Binary Search Tree . . . 5
2.3 Traversing a Binary Tree . . . 5

3 Abstract Data Types in C **6**

4 Your Task **7**
4.1 Create a Plan . . . 7
4.2 Make a Makefile . . . 8
4.3 Write and Test the Functions of the Tree ADT . . . 8
4.3.1 tree_print() . . . 9
4.3.2 tree_alloc() . . . 9
4.3.3 tree_add() . . . 9
4.3.4 tree_free() . . . 9
4.4 Write unique.c . . . 9
4.4.1 check_number() . . . 9
4.4.2 main() . . . 10
4.5 Test Your Program . . . 10
4.6 Review and Update your Comments . . . 11
4.7 add / commit / push / run pipeline / submit commit ID . . . 11

5 Revisions **11**

1 Introduction

In this assignment you are going to write a program, `unique`, that will count the number unique entries in an unordered list of numbers on the command line. You will be using a struct (that we give you in `tree.h`), pointers, pointer operators `*` and `->`, and the dynamic-memory allocation functions `calloc()` and `free()`.

1.1 Get the Resources

☐ **To do:** In a Terminal, synchronize the **resources** repository on your Github Codespaces instance. Then copy the files for **asgn6** into your repository. Remember that the you need to use the `cd` ("change directory") command to set the "current directory" of the Terminal. **The Explorer window does not affect the current directory of the Terminal.**

Note in the first `cd` command below, you want to go to `/workspaces/codespaces-blank` (notice that this "absolute path" starts with a slash `/`), but you can use "auto completion" to reduce the amount of typing that you need to do. That is, instead of typing

```
$ cd /workspaces/codespaces-blank ⏎
```

You can type the first few letters of a directory name followed by a tab key, as demonstrated below.

```
$ cd /w tab /c tab ⏎
$ cd 13s/resources
$ git pull
$ cd ..
$ cp resources/asgn6/* yourcruzid/asgn6
$ cd yourcruzid/asgn6
$ ls
```

Among the files, you should see three executable files (also called "reference binaries" or "demo programs"): `unique_arm`, `unique_hub`, and `unique_x86`. You will be running one of these binaries depending on which microprocessor your Terminal is running on. (The first paragraph of the next section explains this.) After you make your own version of the program, you'll have one more binary: `unique`.

1.2 Run an Example

☐ **To do:** In your repository's `asgn6` directory, you will be running one of the demo programs. Most of you are using Github Codespaces, and so run the command on the left. If you are running Ubuntu on a Mac with Apple silicon, then run the command in the middle. Otherwise, run the command on the right. (If you are not running on GitHub, use the selected commands instead of `./unique_hub` in the box below.)

`./unique_hub` `./unique_arm` `./unique_x86`

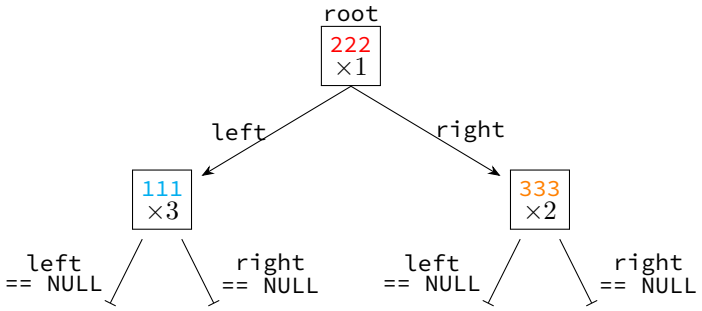
```
$ ./unique_hub 222 333 111 111 333 111 Enter      ⟵ run the program chosen above
3 111     ⟵ program output: one line per unique integer, sorted by value and prefixed by count
1 222
2 333
```

Things to notice about the program output:

- Each number is prefixed by the number of times that it appears in the input.
- Regardless of their input order, the numbers are reported in ascending numerical order (111 → 222 → 333).
- The program's output reports that the input data has three 111s, one 222, and two 333s.

2 About Binary Search Trees

Your program will store data in a *binary tree*. A binary tree is a data structure that has a single root node and that satisfies the requirement that every node of the tree has 0, 1, or 2 children. Below is an example of a binary tree with three nodes. The node with the key value 222 has two children (left and right), and the other two nodes have no children (indicated by `NULL`).



The binary trees that you use in this assignment will be constrained by the values of the nodes' keys. That is, starting at any node, all keys found in the node's left subtree must be less than the node's key (for example, in the figure above, 111 < 222). Also, all keys found in the node's right subtree must be greater than the node's key (for example, in the figure above, 333 > 222). When a binary tree meets these key constraints, it is a *binary* ***search*** *tree*, and it can be used to quickly store and search for keys.

An important observation about binary search trees is that for all sets with more than one key, there are several differently shaped trees that represent the set. For example, sets with three different keys can be stored in five differently shaped trees. Those five trees are the binary search tree above and the four additional trees in Figure 1.

The shape of a binary search tree is determined by the order in which keys are added to the tree. There are six different orders in which three keys can be added to a tree:

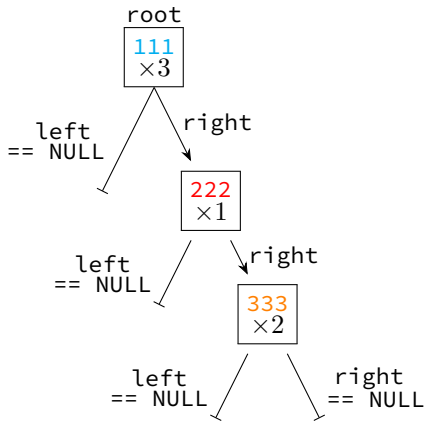
- (111, 222, 333)
- (111, 333, 222)
- (222, 111, 333)
- (222, 333, 111)
- (333, 111, 222)
- (333, 222, 111)

Adding keys in these six different orders creates five different tree shapes: the two orders that start with 222 create the binary search tree above, and the other four orders create the four trees in Figure 1.

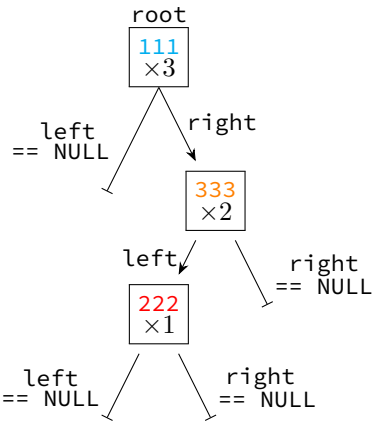
Remember this fact when testing your program. The order of the keys can change the shape of the binary search tree that is generated.

In all five binary search trees, the constraints on the key values still are met.

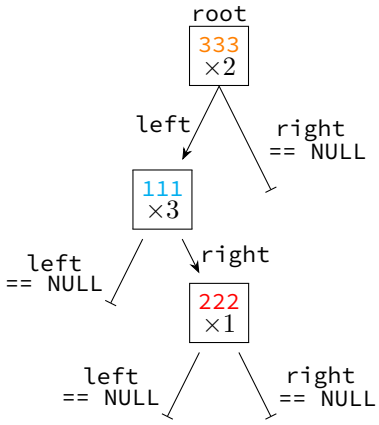
© 2026 Kerry Veenstra



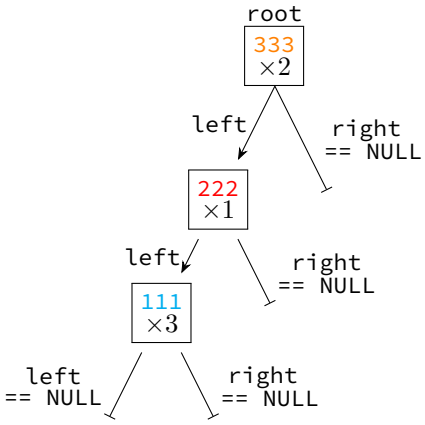
(a) Keys were added in the order 111, 222, 333.



(b) Keys were added in the order 111, 333, 222.



(c) Keys were added in the order 333, 111, 222.



(d) Keys were added in the order 333, 222, 111.

Figure 1: Four binary search trees. Keys of any node's left subtree always are less than the node's key, and keys of any node's right subtree always are greater than the node's key.

2.1 Searching a Binary Search Tree for a Key

You can search any binary search tree for a desired key with iterative steps:

1. Start at the root.
2. Check whether the desired key is greater-than, equal-to, or less-than the node's key.
3. If the keys are equal, your search is done.
4. Otherwise, repeat the check, but with the node's left child node (if the desired key is less than the node's key) or with the node's right child node (if the desired key is greater than the node's key).

☐ **To do:** This is an exercise. You don't need to turn it in. Using the five binary search trees shown earlier, start at the root node and search for the node with key 111. That means going left when the desired key (111) is less than the node's key and going right when the desired key (111) is greater than the node's key. Do you see how the algorithm *always* makes left/right decisions that lead to the desired key? Also do you see that the decisions always follow a downward path?

☐ **To do:** (Another exercise.) Now search for a key that is not in the tree, such as 500. In that case, the search reaches a `NULL` value. So either the algorithm will find the desired key, or its search will be unsuccessful and will end at a `NULL` pointer. That is, if the desired key is not in the tree, then either `root == NULL` (if the tree is empty), or at a left decision `left == NULL`, or at a right decision `right == NULL`.

© 2026 Kerry Veenstra

2.2 Adding a New Node to a Binary Search Tree

One can add a key to a binary search tree using a similar search. However, your program will perform one of two actions depending on whether the tree already contains a node with the desired key.

> If a key search successfully finds a node with the desired key, then increment the node's counter and return. Otherwise, create a new node for the key, and add the node to the tree where the `NULL` pointer was found.

The pseudocode below shows how to add a key to a binary search tree. If the tree lacks the key, then the "pointer to a pointer" `saved` will point to a NULL pointer that needs to be overwritten with a valid pointer to a new node.

```
tree_add(tree, desired_key):
    saved = &(tree->root)
    current_node = tree->root

    while current_node != NULL:
        if desired_key == current_node->key:
            current_node->count += 1
            return
        if desired_key < current_node->key:
            saved = &(current_node->left)
            current_node = current_node->left
        if desired_key > current_node->key:
            saved = &(current_node->right)
            current_node = current_node->right

    // The tree has no node with the desired key.
    create a new node (the pointer new_node points to the new node)
    new_node->key = desired_key
    new_node->count = 1

    // Finally, use the saved "pointer to a pointer" to add to the tree.
    *saved = new_node
```

Being able to overwrite a `NULL` pointer of the tree that is encountered during the search is a critical part of your algorithm. That is what the pointer `saved` is for.

2.3 Traversing a Binary Tree

> "Traversing" a tree means visiting all of its nodes.

There are many kinds of traversal algorithms. `unique` is going to use two of them: an "in-order" traversal (for printing a tree) and a "post-order" traversal (for freeing a tree).

Since a binary tree is a recursive data type, you can traverse the entire tree using a recursive function (a function that calls itself). An "in-order" traversal is shown below. You would call the function, passing the root of the tree as the node parameter. "Visiting" a node means doing something with the node, such as printing it.

```
traverse_in_order(node):
    If the node is not NULL, then
        traverse_in_order(node->left)
        visit the node (print it)
        traverse_in_order(node->right)
```

The "in" part of "in-order traversal" means that the "visiting" part of the algorithm is performed in-between the recursive function calls. Remember that our definition of a binary tree requires that all keys of the left subtree come before the key of the subtree's root? That means that by recursing down the left child before visiting the subtree's root, all of the lower keys will be printed before the root's key is printed. The same argument goes for the right subtree. Therefore, all keys will be printed in sorted order.

Our traversal algorithm's "visiting" part could be placed at two other places: after the two recursive function calls or before them. Below are algorithms that do "post-order" and "pre-order" traversals. Compare them with each other and with the in-order traversal. Notice that the left subtree always is traversed before the right subtree, but each of the algorithms visits the node at different times.

```
traverse_post_order(node):
    If the node is not NULL, then
        traverse_post_order(node->left)
        traverse_post_order(node->right)
        visit the node (free it)
```

```
traverse_pre_order(node):
    If the node is not NULL, then
        visit the node
        traverse_pre_order(node->left)
        traverse_pre_order(node->right)
```

So the node is visited before both recursive calls with a "pre-order" traversal, the node is visited after both recursive calls with a "post-order" traversal, and the node is visited between the recursive calls with an "in-order" traversal.

- **in-order traversal:** You will use this when printing the nodes of a binary search tree in sorted order.
- **post-order traversal:** You will use this when freeing the nodes of a tree.
- **pre-order traversal:** Not used in this assignment, but can be used when copying a binary tree.

3 Abstract Data Types in C

The C programming language supports a few native data types such as `int`, `char`, and `double`. We can use variables of these data types directly in expressions and function calls.

C also lets us construct complicated data types like trees, but these data types cannot appear directly in expressions or function calls. So to make it easier to use these complicated data types, one provides several functions to access or manipulate the complicated data type. We call such a collection of types and functions an "abstract data type" (ADT).

An Abstract Data Type (ADT) groups one or more type definitions with one or more function definitions. Sometimes the ADT provides little or no direct access to the ADT's internal variables.

The ADT for our binary search tree is defined in the file `tree.h`. Within that file, you will see two type definitions and five function definitions. First, there is the definition of the `Node` data type. Each `Node` has four parts: the node's `key`, a `count` of the number of times that that key was seen, and two pointers to the `left` and `right` children of the node (which are `Node`s themselves). If a node has no left child, then its `left` pointer is NULL; if the node has no right child, then its `right` pointer is NULL.

```
typedef struct s_node Node;      ⟵ the type is called Node (also struct s_node)

struct s_node {
    int key;                     ⟵ the integer key
    int count;                   ⟵ the number of times that the key was seen
    Node *left;                  ⟵ pointer to the left child or NULL
    Node *right;                 ⟵ pointer to the right child or NULL
};
```

Next is the definition of the `Tree` data type. It has only a single field, which is a pointer to the `root` node of a tree. Why do we define a second structure for a tree if we already have a structure for a node? Because an empty tree is represented by a NULL `root` field, and we still need a data structure to store the NULL pointer.

```
typedef struct s_tree Tree;      ⟵ the type is called Tree (also struct s_tree)

struct s_tree {
    Node *root;                  ⟵ a Tree has a single root Node.
};
```

Finally, here are the function definitions for the binary search tree ADT. These functions let you create ("alloc") a new tree and destroy ("free") it. You also can add a new key to the tree and print the tree. A final function lets you dump a visual representation of the tree for debugging purposes.

```
Tree *tree_alloc(void);
void tree_add(Tree *tree, int key);
void tree_print(Tree *tree);
void tree_free(Tree **p);
void tree_dump(Tree *tree);
```

You can define more functions that you find useful.

4 Your Task

4.1 Create a Plan

☐ **To do:** Skim the rest of this document and make notes about what you need to do. In particular, pay attention to sections 4.3 and 4.4.
☐ **To do:** Read through the starter file `tree.c`. Notice that there are eight functions. You will be writing new "bodies" for six of them.
☐ **To do:** Read through the starter file `unique.c`. Notice that there are three functions. You will be writing new "bodies" for two of them.

4.2 Make a Makefile

Why do you use a `Makefile`? Because you need to make two executable files: `unique` and `unittests`.

Unlike some other programming languages, if you have *multiple* C program files, you create an executable file through multiple intermediate "object files" (`unique.o` and `tree.o` are combined to make `unique`, and `unittests.o` and `tree.o` are combined to make `unittests`). The steps are shown below. These are what we want the `make` command to do. (Add the `-gdwarf-4` option to the `clang` command to make it possible to use a debugger.)

In the `Makefile` define a variable:

```
CFLAGS = -Wall -Werror -gdwarf-4    ⟵ define the CFLAGS variable for later use
```

These actions should appear in your `Makefile`, ***each after an appropriate dependency line***:

```
clang -c $(CFLAGS) tree.c           ⟵ create the "object file" tree.o from tree.c
clang -c $(CFLAGS) unique.c         ⟵ create the "object file" unique.o from unique.c
clang -c $(CFLAGS) unittests.c      ⟵ create the "object file" unittests.o from unittests.c
clang unique.o tree.o -o unique     ⟵ create "executable file" unique from .o files
clang unittests.o tree.o -o unittests    ⟵ create "executable file" unittests from .o files
```

☐ **To do:** Create the file `Makefile` to make the `unique` and `unittests` executables.

- Remember that Assignment 3 and the Lectures on October 28th demonstrated using a `Makefile`.
- Remember that the `make` program will make the **first** target that is listed in your `Makefile`, along with any other dependent files. (The "target" is underlined in the dependency line below.) So if the first dependency line of a `Makefile` is the one below, then the `Makefile` will execute commands to make `unique` and `unittests` (which is what you want).

  When looking at a `Makefile` dependency line, remember, "this *depends on* that ."

  ```
  .PHONY: all     ⟵ all doesn't get made, and ignore the date of any file named all
  all: unique unittests     ⟵ all depends on unique and unittests
  ```

☐ **To do:** Run your `Makefile` to compile `unique` and `unittests`.

Running your `Makefile`

```
$ make     ⟵ That's all that's needed.
```

4.3 Write and Test the Functions of the Tree ADT

You will be writing functions and running "unit tests" (that is, tests that test just the functions that you just wrote). Look in `tree.c` for the details about each of these first six functions. Review section 3, which discusses the definitions in `tree.h`. I also ask you to run some tests.

If a test fails, then you will need to open `unittests.c`, examine the `switch` statement at the end to determine which test function failed, and then examine the code in the function to identify what the test is doing.

Example: let's say that Test 5 fails. For Test 5, the `switch` statement calls `test_print_three_nodes()`. That function creates a (correct) balanced three-node tree and then calls your `tree_print()` function. If the output is unexpected, then the problem resides in `tree_print()` or `tree_print_node()`.

4.3.1 tree_print()

☐ **To do:** Write `tree_print()`.
☐ **To do:** Write `tree_print_node()`.
☐ **To do:** Confirm that the first six unit tests pass when you run `./unittests 6`.

```
$ ./unittests 6 Enter        ⟵ run the first six tests
Running 6 of 18 tests

test 1:
expect no output
-----
-----

test 2:
expect one line "1 100"
-----
1 100
-----

(omitting additional output)
```

4.3.2 tree_alloc()

☐ **To do:** Write `tree_alloc()`.
☐ **To do:** Confirm that the first seven unit tests pass when you run `./unittests 7`.

4.3.3 tree_add()

☐ **To do:** Write `tree_add()`.
☐ **To do:** Confirm that the first 12 unit tests pass when you run `./unittests 12`.

4.3.4 tree_free()

☐ **To do:** Write `test_free()` and `test_free_node()`.
☐ **To do:** Confirm that all 18 unit tests pass when you run `./unittests 18`.

4.4 Write unique.c

4.4.1 check_number()

Here is the pseudocode for the function `check_number()`.

```
check_number(s):
    for every char ch in string s do:
        if ch is not a digit then
            call print_usage()
```

☐ **To do:** Write a function-header comment for the function `check_number()`. Remember to include these parts:

- **Purpose**: What is the function's purpose?
- **Parameters**: What are the names, types, and uses of the functions input parameters? (Or if appropriate, mention that there are none.)
- **Returns**: What is the type and purpose of the function's return value? (Or if appropriate, mention that there is none.)

☐ **To do:** Write some comments in the body of the function `check_number()`, documenting your ideas on the C code that you will write.
☐ **To do:** Write `check_number()`.

4.4.2 main()

Here is the pseudocode for the function `main()`.

```
    main():
        allocate a tree

        for i in (1 ... argc - 1)
            if argv[i] is "-d" then
                skip this argument, but note that -d was seen
            else
                check_number(argv[i]);
                tree_add(tree, atoi(argv[i]));

        if -d was seen then
            tree_dump()
        else
            tree_print()

        free the tree
```

☐ **To do:** Write a function-header comment for the function `main()`.
☐ **To do:** Write some comments in the body of the function `main()`, documenting your ideas on the C code that you will write.
☐ **To do:** Write `main()`.

4.5 Test Your Program

To receive full credit, the output of your program must match the output of a reference program that we provide. You can use the `diff` program as you did in prior assignments to compare the output of the reference program and the output of your program.

Here are two examples of running tests, but you should devise and run several more.

```
./unique_hub 222 111 333 > expected.txt
./unique     222 111 333 > got.txt
diff -us expected.txt got.txt
```

```
./unique_hub 333 222 111 > expected.txt
./unique     333 222 111 > got.txt
diff -us expected.txt got.txt
```

Optional: You can automate your testing by creating a `bash` shell script `test.sh` that executes those three commands. (See Lectures 4 and 5.) To make the script useful with any test input, remember that in a `bash` script, the `$1`, `$2`, and `$3` refer to the first three arguments when running the `./test.sh 222 111 333` command. But even better would be to support any number of arguments and not just three. So remember that in a `bash` script `$@` refers to all of the arguments! **In the script**, a line like this would be useful:

```
./unique $@ > expected.txt
```

© 2026 Kerry Veenstra

You can use the `-d` option to generate a textual representation of a binary tree, as shown below. The root is on the left. Tilt your head to the left to see the tree. (The actual tree that the output represents is in Figure 2.)

```
$ ./unique -d 222 333 111 111 333 111 Enter      ⟵ run the program with "-d"
        / NULL     ⟵ program output: representation of Figure 2 (tilted to the left)
    / 333 (x2)
        \ NULL
< 222 (x1)
        / NULL
    \ 111 (x3)
        \ NULL
```

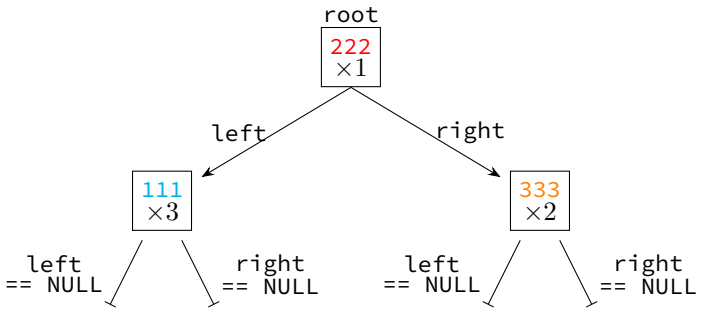


Figure 2: A binary tree.

4.6 Review and Update your Comments

☐ **To do:** Confirm that the comments of your functions `check_number()` and `main()` are consistent with the C code.

4.7 add / commit / push / run pipeline / submit commit ID

You need to submit these files:

- `Makefile`
- `unique.c`
- `tree.c`
- `unittests.c`
- `tree.h`

☐ **To do:** `git add` all of the files above.
☐ **To do:** `git commit -m '`*the changes that you made*`'` (you can give any reason for this commit)
☐ **To do:** `git push`
☐ **To do:** Run the `asgn6` pipeline at **git.ucsc.edu**.
☐ **To do:** Submit your final commit ID to Canvas by Wednesday November 18th at 11:30 pm.

5 Revisions

Version 1 Original.

© 2026 Kerry Veenstra